

Graph Neural Networks: Perspective

Jiaxuan You

Assistant Professor at UIUC CDS



CS512: Data Mining Principles, 2025 Fall

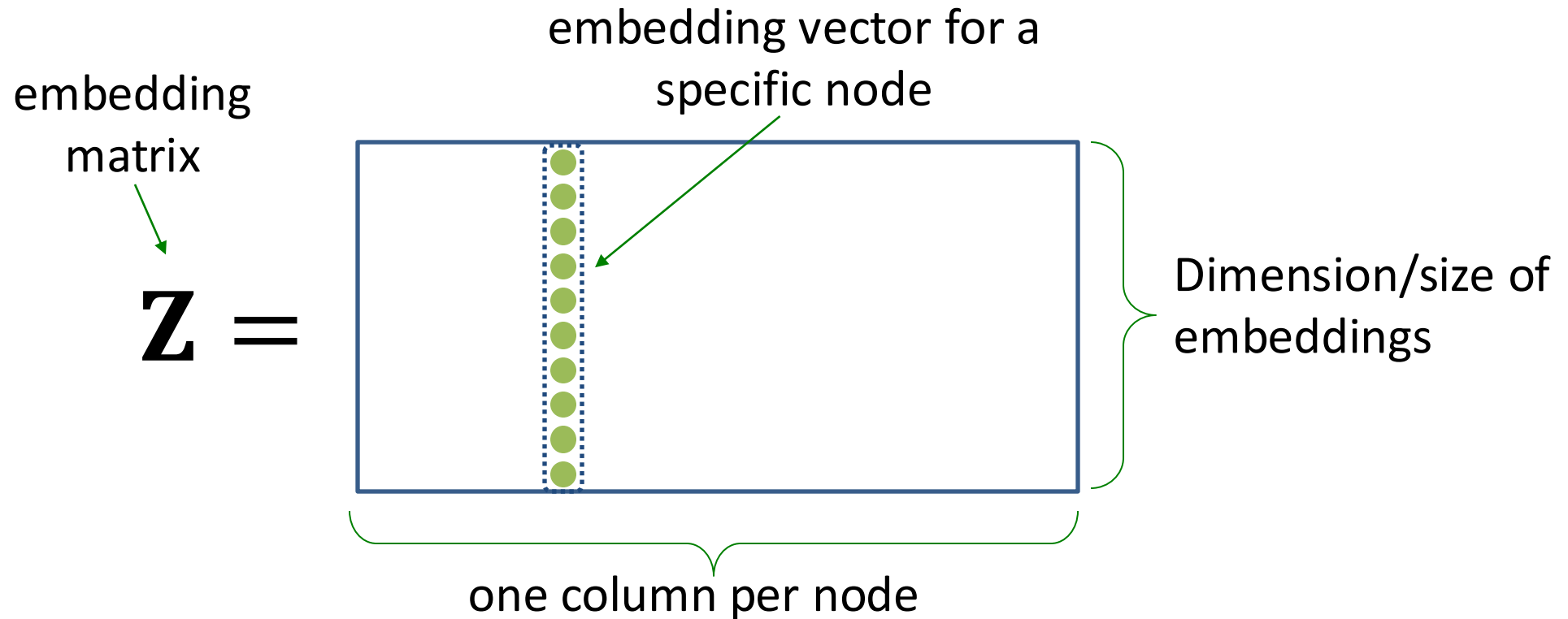
<https://ulab-uiuc.github.io/CS512/>

Announcements

- **Reminder:** Writing task due **Sept 17 11:59pm**
 - [Template](#) has been released on Canvas
- Course project:
 - A group of **3 students is expected**. You may start finding teammates on Slack.
 - If you have solid reasons to form a group of 2 or 4, please let our TA know
 - Expect collaborations: Every member will submit a brief *summary of contributions* in final submission
- Topics: any research idea related to graphs. Some references
 - Leverage your insights from **writing tasks, idea proposal, and discussions**
 - We will also suggest some potential project ideas
 - <https://logconference.org/cfp/>, <https://glfrontiers.github.io/2023/cfp/>
 - Focus on ideas that need low resources (Google Colab / laptop / free LLM tools). Proof-of-concept style of experiment will be sufficient

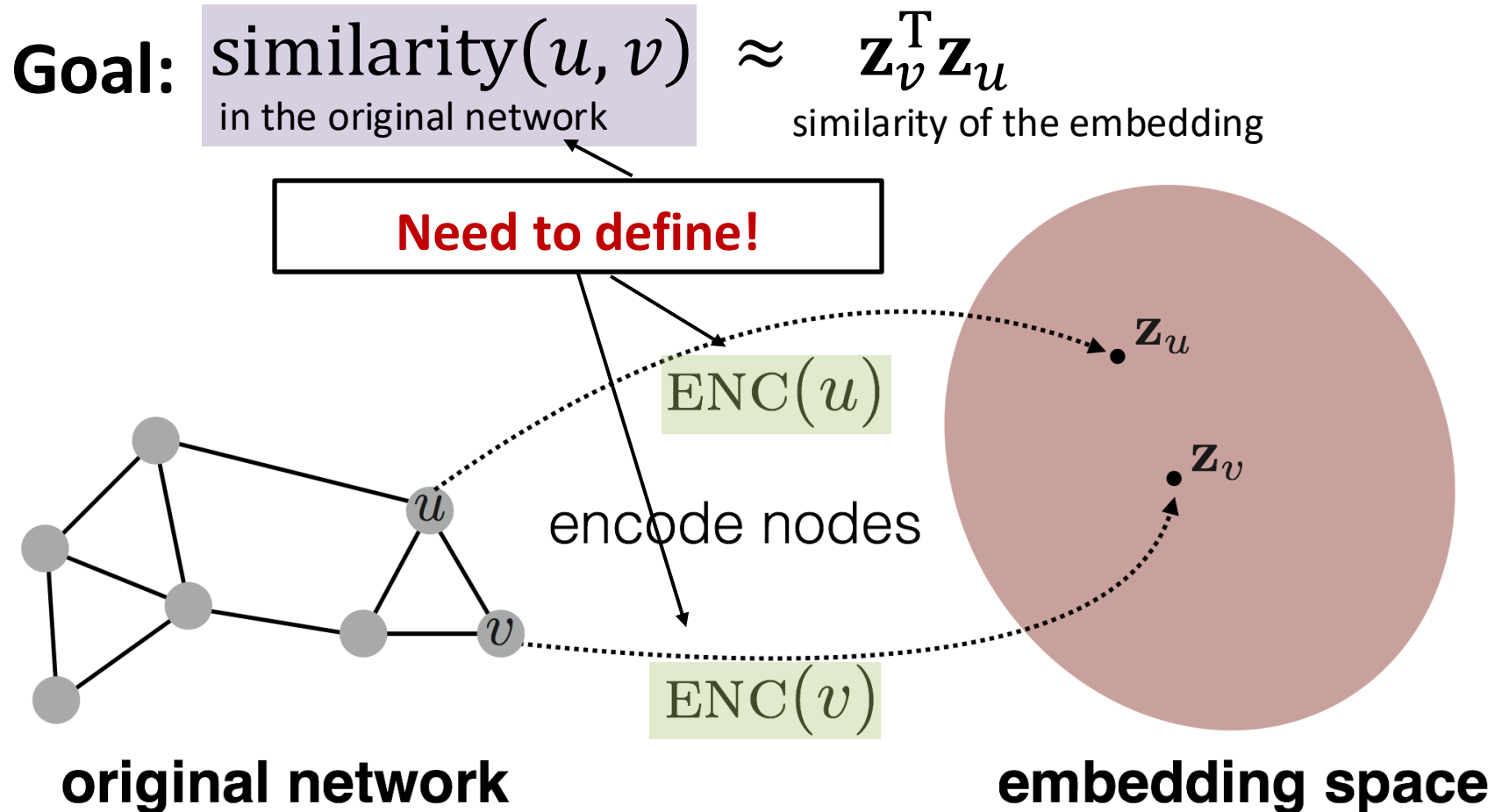
Recap: “Shallow” Encoding

- Simplest encoding approach: **encoder is just an embedding-lookup**



Recap: Similarity Based Objective Function

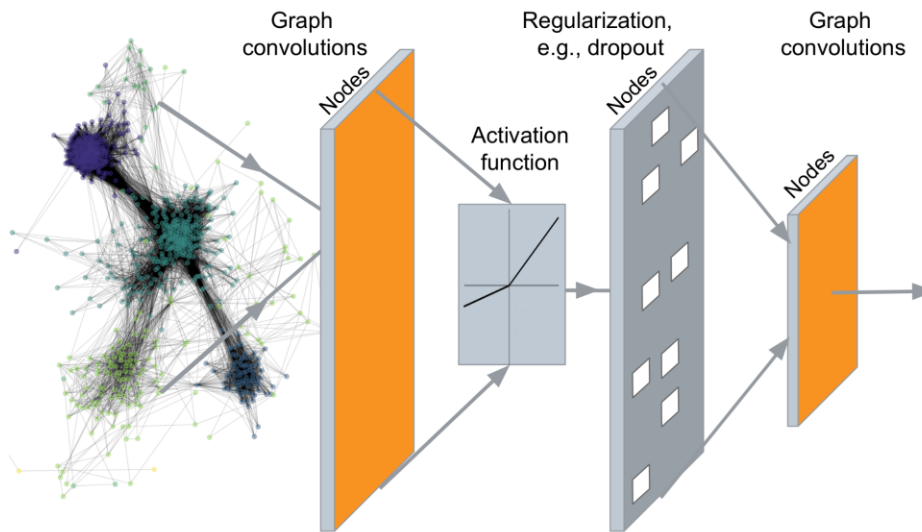
- Idea of **self-supervised learning**. Requires an **edge-level prediction head**



Today: Deep Graph Encoders

- **Today**: More sophisticated encoder, compared to embedding lookup -- **graph neural networks (GNNs)**:

$ENC(v) =$
multiple layers of
non-linear transformations
based on graph structure



Output: Node embeddings + node/edge/graph prediction heads

- **The learning pipeline is unchanged:** Still output node embeddings, and can be combined with different **node/edge/graph prediction heads**

Graph Neural Networks: Perspective

Deep Learning for Graphs

Content

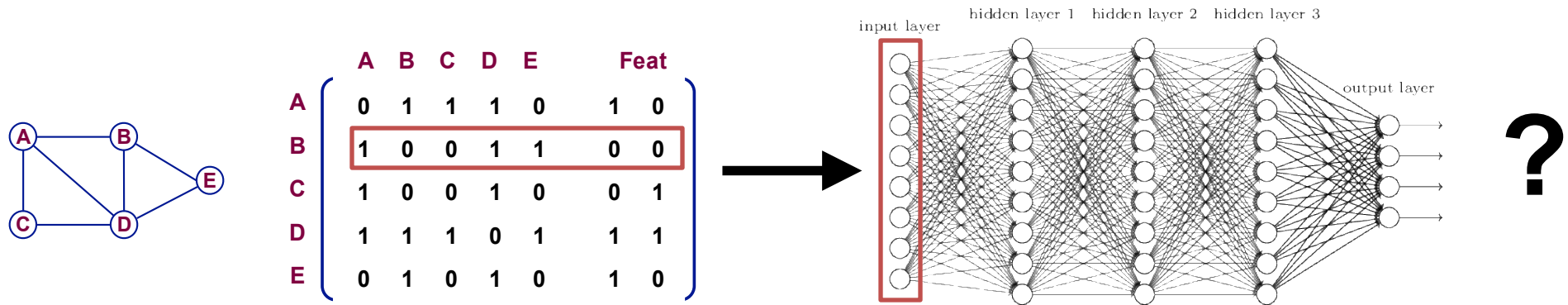
- **Local network neighborhoods:**
 - Describe aggregation strategies
 - Define computation graphs
- **Stacking multiple layers:**
 - Describe the model, parameters, training
 - How to fit the model?
 - Simple example for unsupervised and supervised training

Setup

- Assume we have a graph G :
 - V is the **vertex set**
 - A is the **adjacency matrix** (assume binary)
 - $X \in \mathbb{R}^{|V| \times d}$ is a matrix of **node features**
 - **Edge features** could be added; Or we can **project a graph to bipartite graph to convert edges to nodes**
 - v : a node in V ; $N(v)$: the set of neighbors of v .
 - **Node features:**
 - From the specific domains:
 - Social networks: User profile, User image
 - Biological networks: Gene expression profiles, gene functional information
 - When there is no node feature in the graph dataset:
 - Indicator vectors (one-hot encoding of a node)
 - Vector of constant 1: $[1, 1, \dots, 1]$

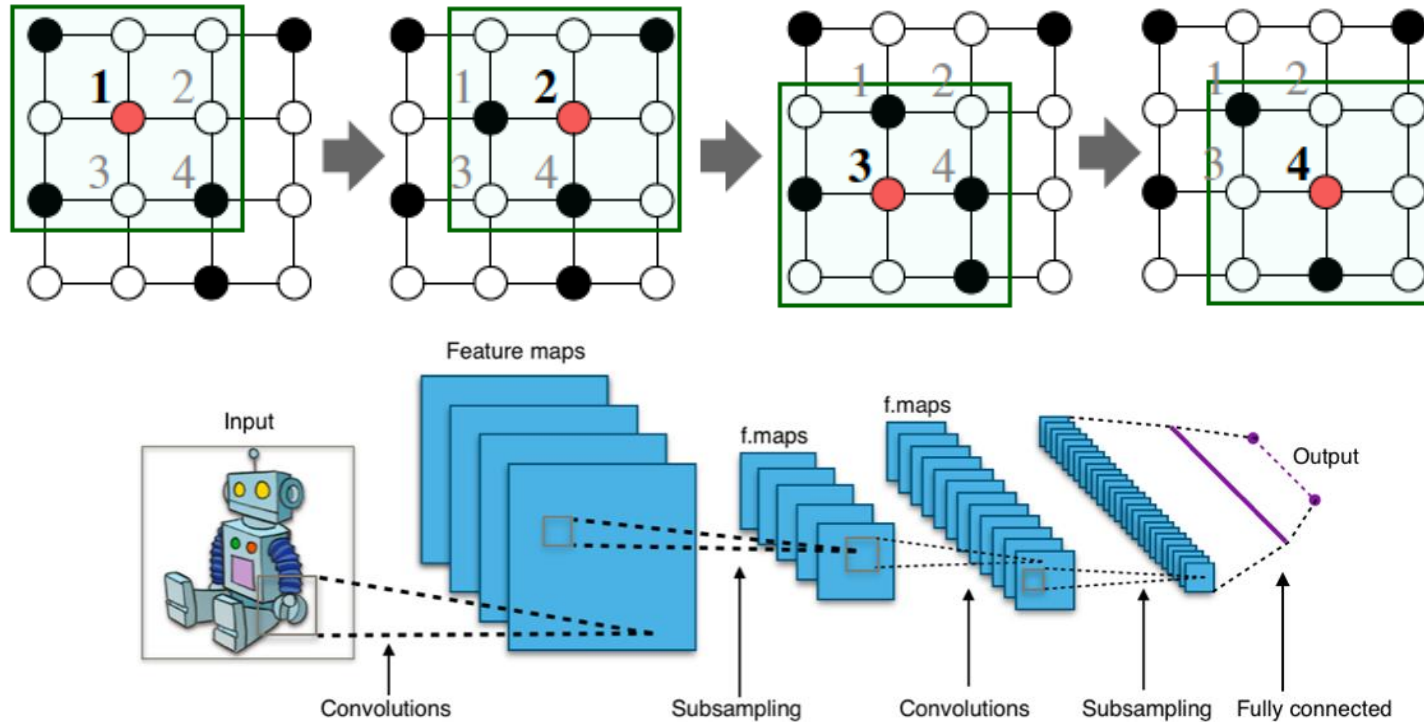
A Naïve Approach

- Join adjacency matrix and features
- Feed them into a deep neural net:



- Issues with this idea:
 - $O(|V|)$ parameters
 - Not applicable to graphs of different sizes
 - Sensitive to node ordering

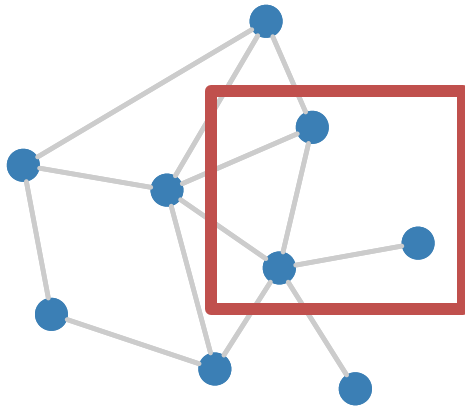
Idea: Convolutional Networks



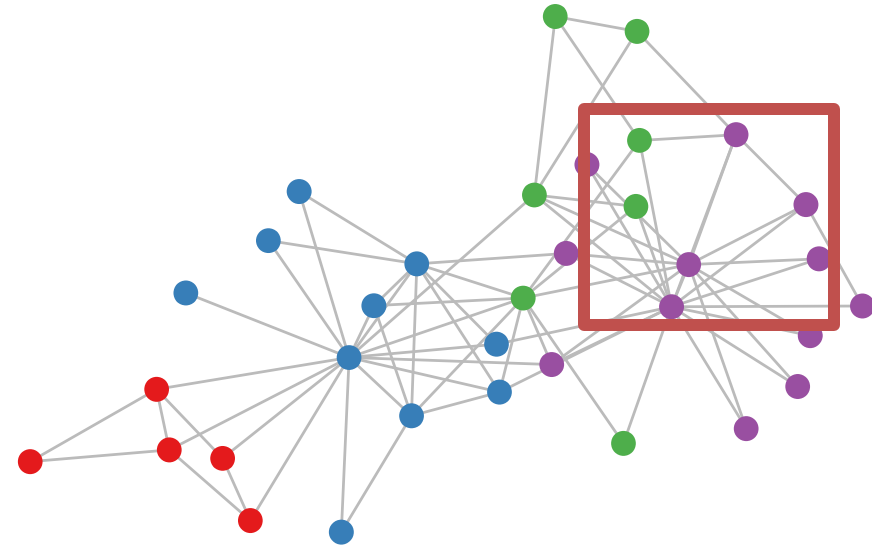
Goal is to generalize convolutions beyond simple lattices
Leverage node features/attributes (e.g., text, images)

Real-World Graphs

- But our graphs look like this:



or this:



- There is no fixed notion of locality or sliding window on the graph
- Graph is **permutation invariant**

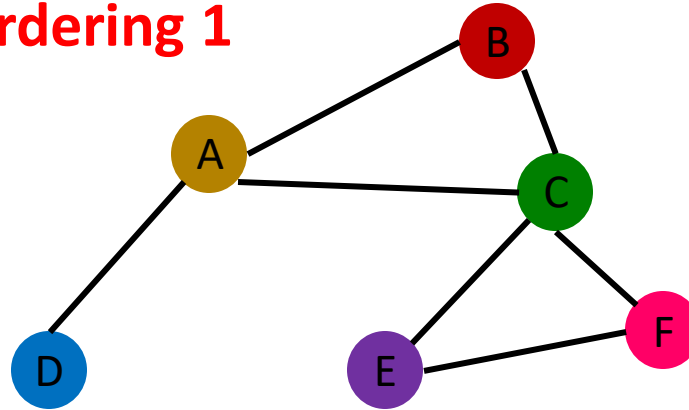
Graph Property 1: Permutation Invariance

- **Graph does not have a canonical order of the nodes!**
- We can have many different node orders.

Permutation Invariance

- Graph does not have a canonical order of the nodes!

Ordering 1



Node features X_1

A	
B	
C	
D	
E	
F	

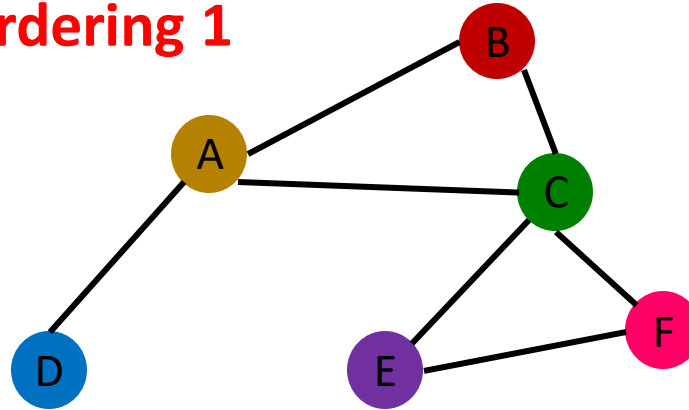
Adjacency matrix A_1

	A	B	C	D	E	F
A						
B						
C						
D						
E						
F						

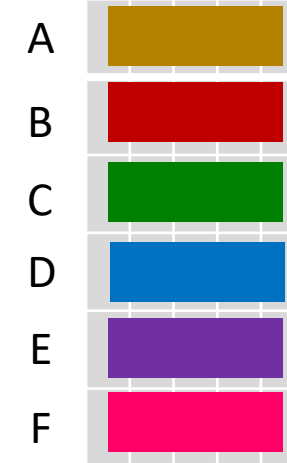
Permutation Invariance

- Graph does not have a canonical order of the nodes!

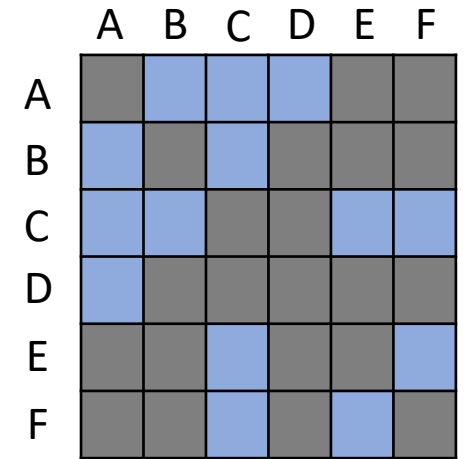
Ordering 1



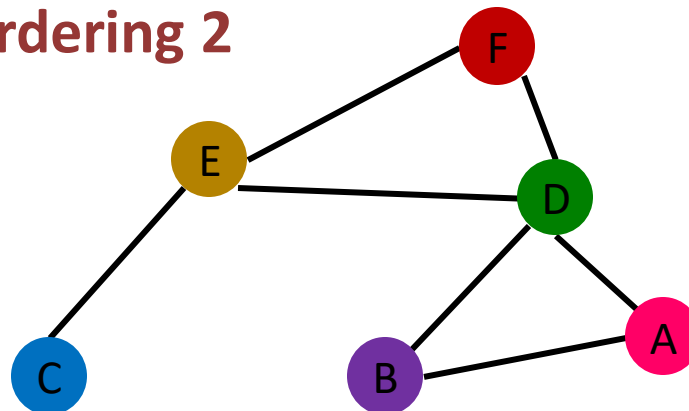
Node features X_1



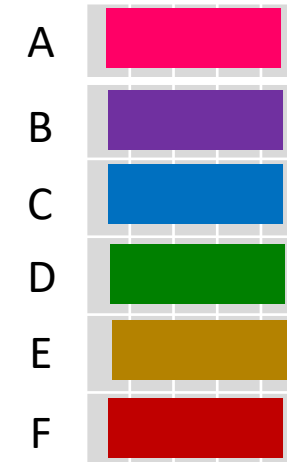
Adjacency matrix A_1



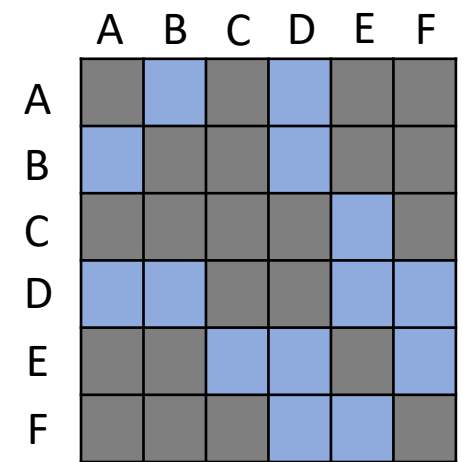
Ordering 2



Node features X_2



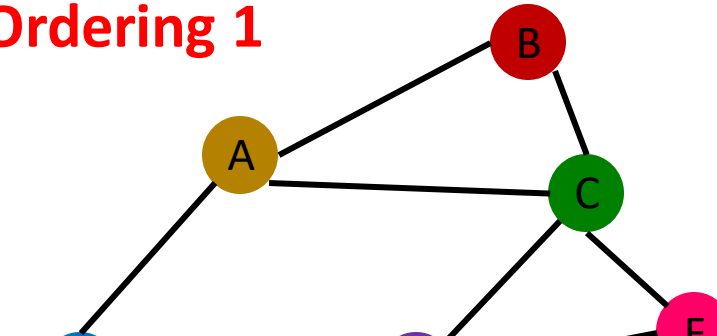
Adjacency matrix A_2



Permutation Invariance

- Graph does not have a canonical order of the nodes!

Ordering 1



Node features X_1

A	
B	
C	
D	
E	
F	

Adjacency matrix A_1

	A	B	C	D	E	F
A						
B						
C						
D						
E						
F						

Ordering 2

Graph representations should be the same for **Ordering 1** and **Ordering 2**

Graph diagram for Ordering 2. Nodes are labeled C (blue), B (purple), A (pink), D (green), E (yellow), and F (red). Edges connect (C,B), (B,A), (A,D), (A,E), and (D,E).

C	
D	
E	
F	

	C	D	E	F
C				
D				
E				
F				

Permutation Invariance

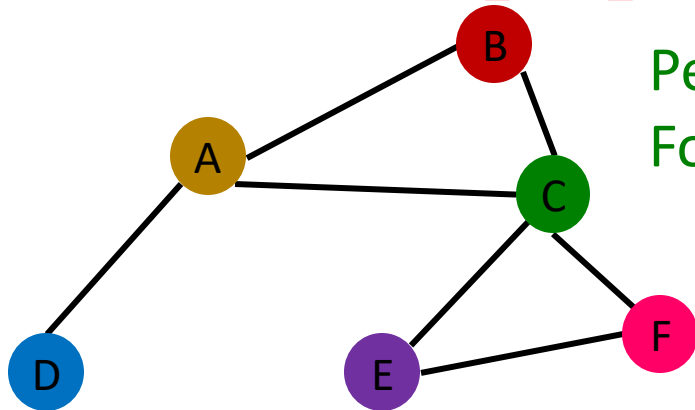
Graph representations should be the same for different orderings

- Graph embedding: Consider we learn a function f that maps a **graph** $G = (A, X)$ to a **vector** $\mathbb{R}^d$ then

$$f(A_1, X_1) = f(A_2, X_2)$$

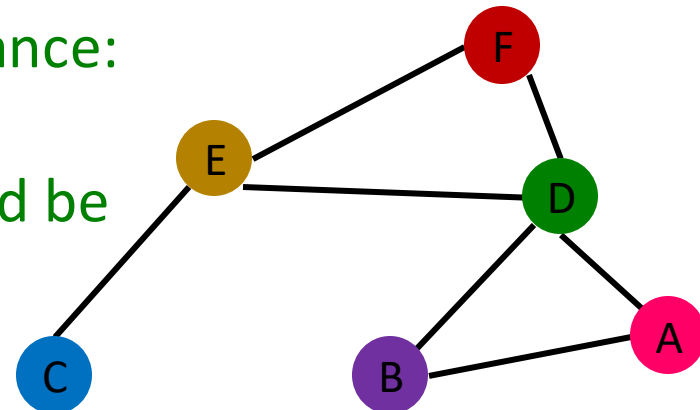
A is the adjacency matrix
 X is the node feature matrix

Ordering 1: A_1, X_1



Permutation invariance:
For two orderings,
output of f should be
the same!

Ordering 2: A_2, X_2



Permutation Invariance

Graph representations should be the same for different orderings

- Consider we learn a function f that maps a graph $G = (A, X)$ to a vector $\mathbb{R}^d$.
- Then, if $f(A_i, X_i) = f(A_j, X_j)$ for any order plan i and j , we formally say f is a **permutation invariant function**.
- **Definition:** For any graph function $f: \mathbb{R}^{|V| \times m} \times \mathbb{R}^{|V| \times |V|} \rightarrow \mathbb{R}^d$, f is **permutation-invariant** if $f(A, X) = f(PAP^T, PX)$ for any permutation P .

A is the adjacency matrix
 X is the node feature matrix

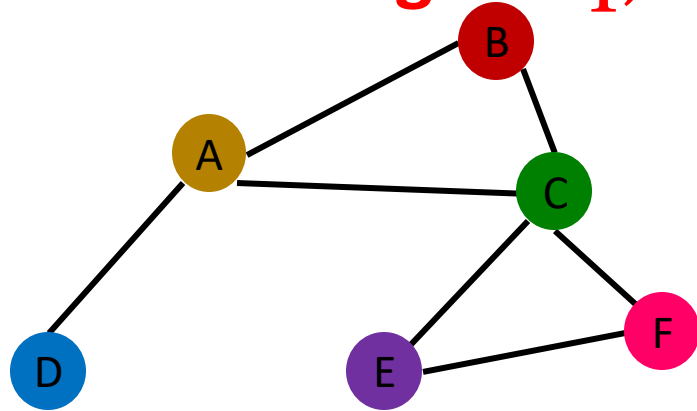
For a graph with $|V|$ nodes,
there are $|V|!$ different
order plans.

Permutation P : a shuffle of
the node order
Example: (A,B,C) $\rightarrow$ (B,C,A)

Graph Property 2: Permutation Equivariance

- Node representation should also be consistent for different orderings
- We learn a function f that maps nodes of G to a matrix $\mathbb{R}^{m \times d}$.

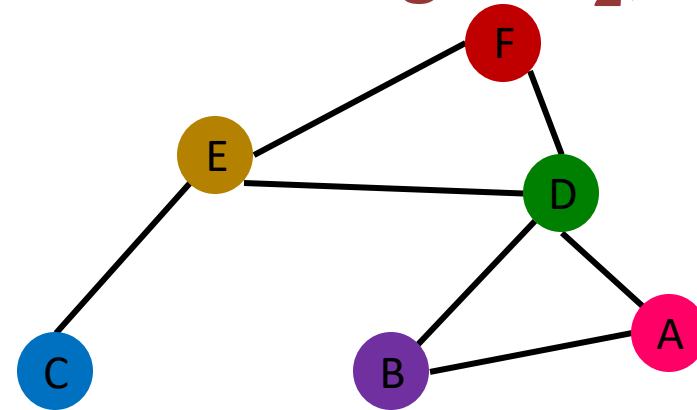
Ordering 1: A_1, X_1



$f(A_1, X_1) =$

A		
B		
C		
D		
E		
F		

Ordering 2: A_2, X_2



$f(A_2, X_2) =$

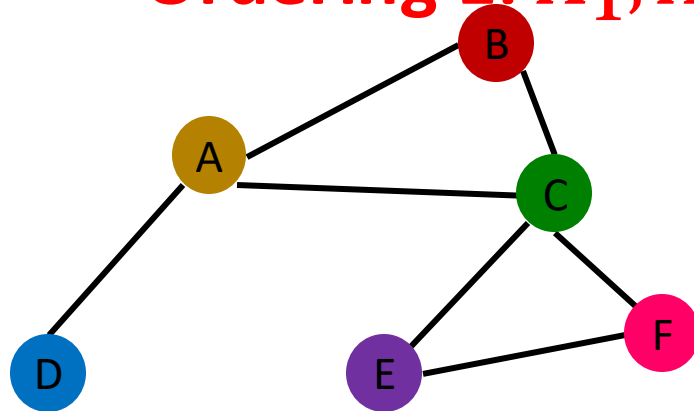
A		
B		
C		
D		
E		
F		

What do we expect here?

Permutation Equivariance

- Node representation should also be consistent for different orderings
- We learn a function f that maps nodes of G to a matrix $\mathbb{R}^{m \times d}$.

Ordering 1: A_1, X_1

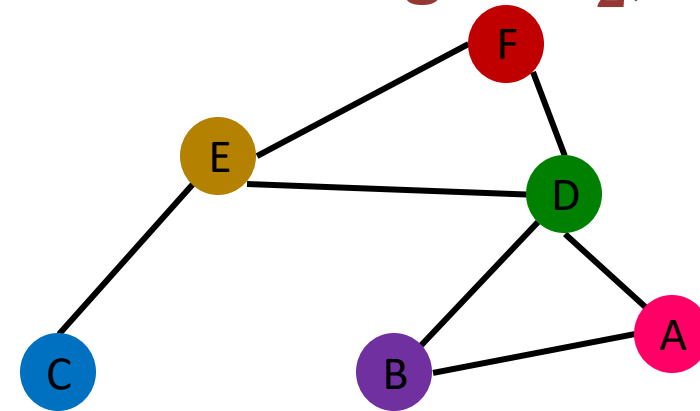


$f(A_1, X_1) =$

A		
B		
C		
D		
E		
F		

Representation vector of the brown node A

Ordering 2: A_2, X_2



$f(A_2, X_2) =$

A		
B		
C		
D		
E		
F		

Representation vector of the brown node E

For two order plans, the **node embedding for the same node** in the graph is the same!

Permutation Equivariance

For node representation

- Consider we learn a function f that maps a graph $G = (A, X)$ to a matrix $\mathbb{R}^{m \times d}$
- If the output vector of a node at the same position in the graph remains unchanged for any order plan, we say f is **permutation equivariant**.
- **Definition:** For any **node** function $f: \mathbb{R}^{|V| \times m} \times \mathbb{R}^{|V| \times |V|} \rightarrow \mathbb{R}^{|V| \times m}$, f is **permutation-equivariant** if $Pf(A, X) = f(PAP^T, PX)$ for any permutation P .

Summary: Invariance and Equivariance

- **Graph Property 1: Graph-level permutation-invariant**

$$f(A, X) = f(PAP^T, PX)$$

Permute the input, the output stays the same.
(map a graph to a vector)

- **Graph Property 2: Node-level Permutation-equivariant**

$$Pf(A, X) = f(PAP^T, PX)$$

Permute the input, output also permutes accordingly.
(map a graph to a matrix)

- **Examples:**

- $f(A, X) = \mathbf{1}^T X$: Permutation-**invariant**

- Reason: $f(PAP^T, PX) = \mathbf{1}^T PX = \mathbf{1}^T X = f(A, X)$

- $f(A, X) = X$: Permutation-**equivariant**

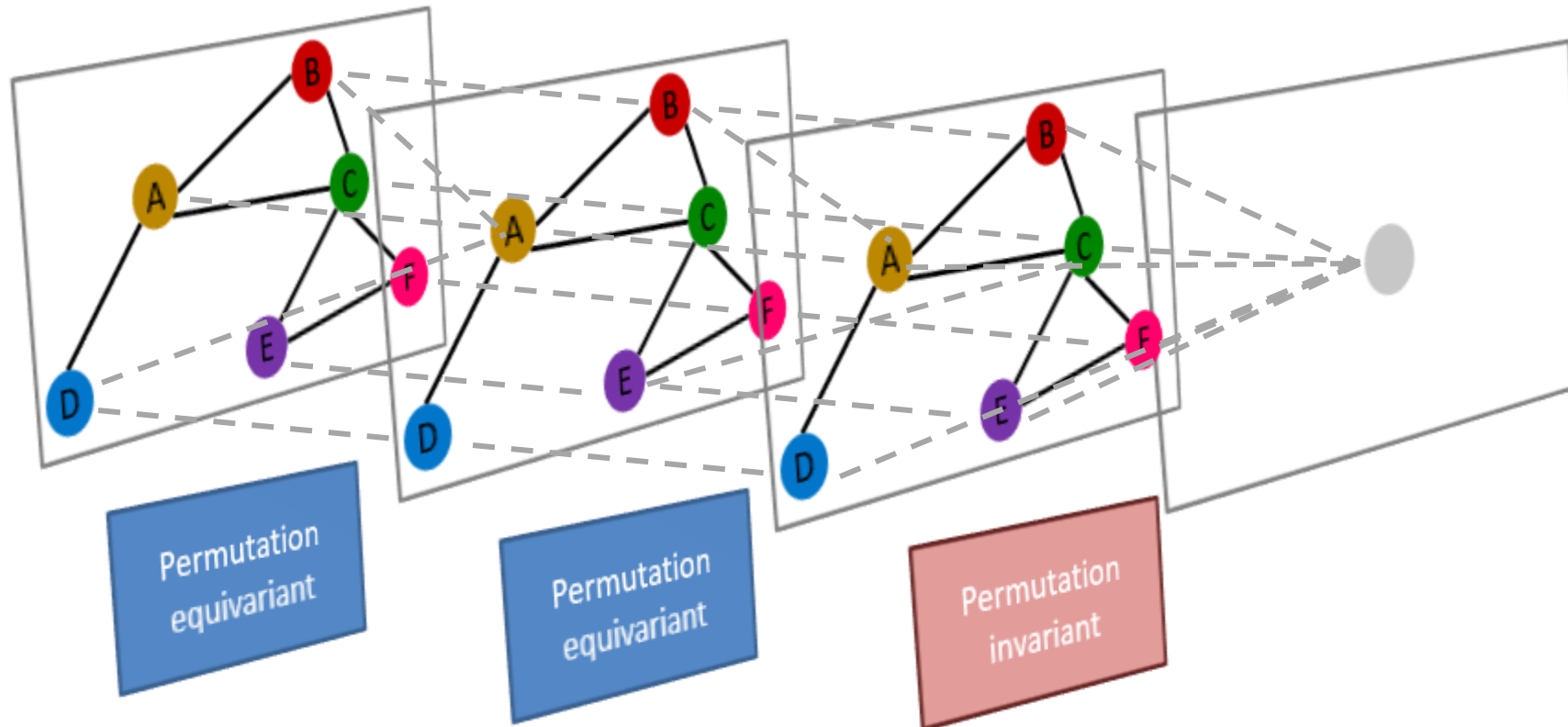
- Reason: $f(PAP^T, PX) = PX = Pf(A, X)$

- $f(A, X) = AX$: Permutation-**equivariant**

- Reason: $f(PAP^T, PX) = PAP^T PX = PAX = Pf(A, X)$

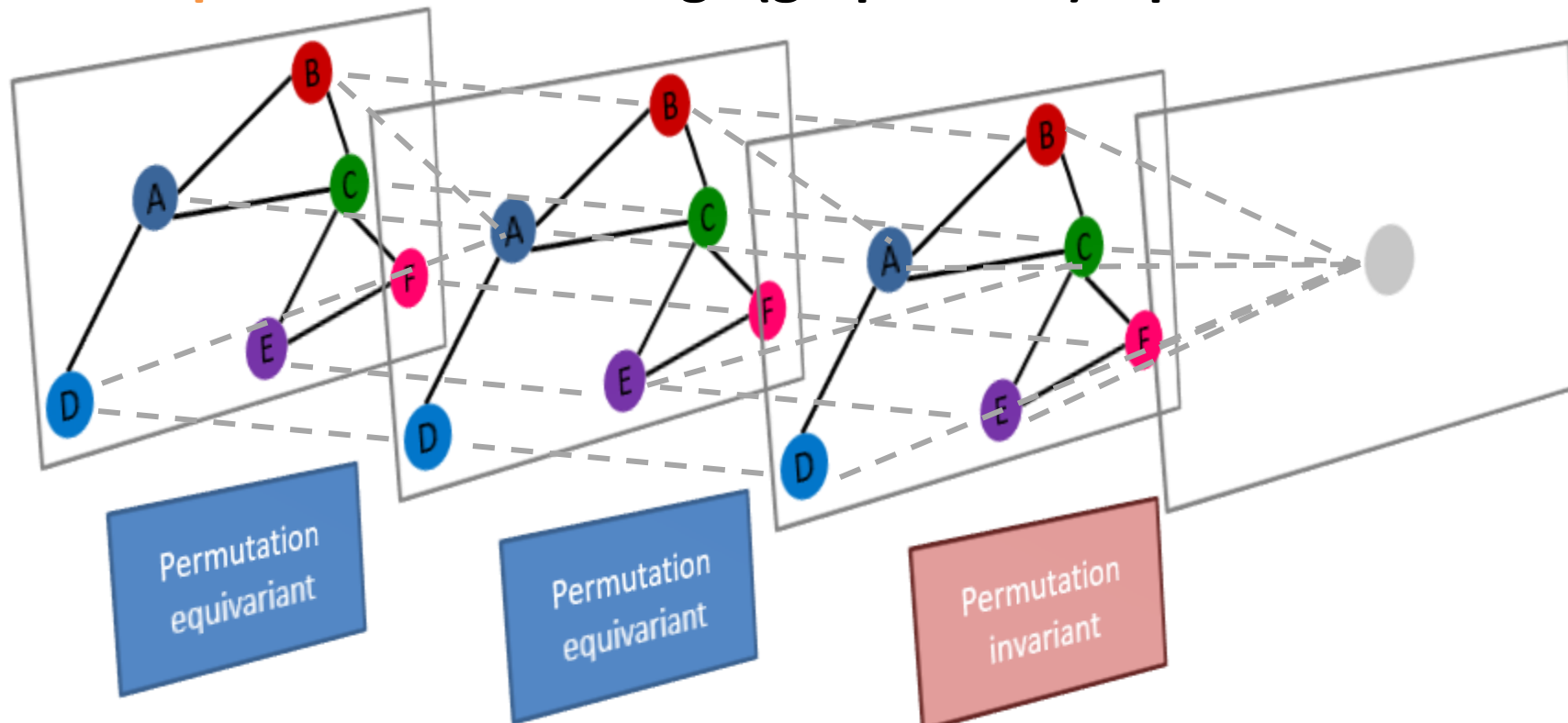
Graph Neural Network Overview

- Graph neural networks (GNNs) consist of **multiple permutation equivariant / invariant functions** for different GNN layers



Graph Neural Network Overview

- GNN layer take nodes as input
 - When **output multiple** embeddings (node-level) – permutation **equivariant**
 - When **output one** embeddings (graph-level) – permutation **invariant**

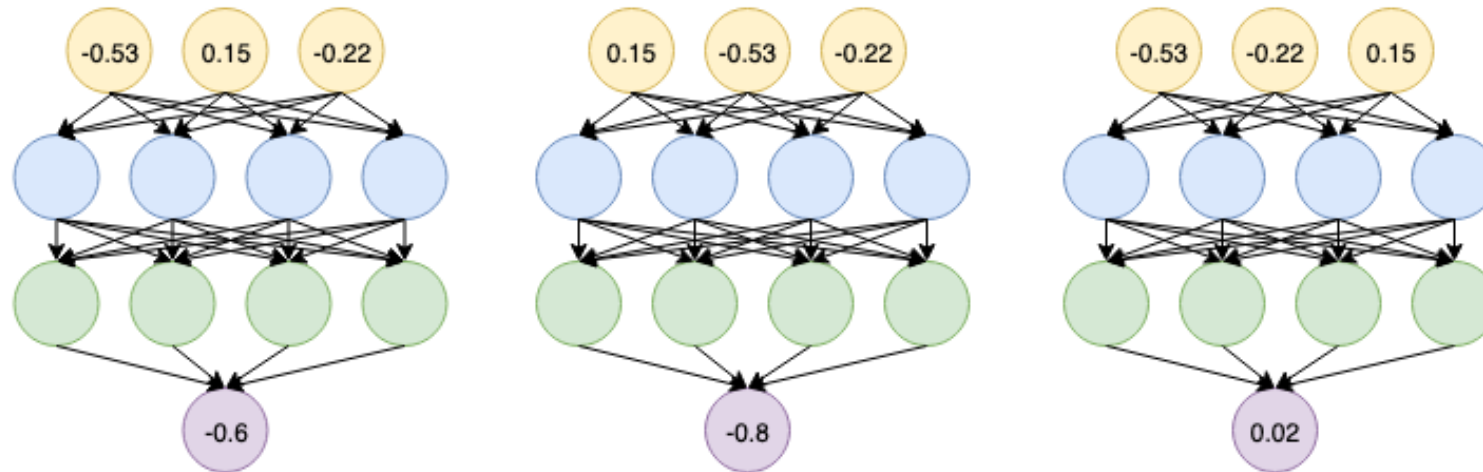


Graph Neural Network Overview

Are other neural network architectures, e.g., MLPs, permutation invariant / equivariant?

■ **No.**

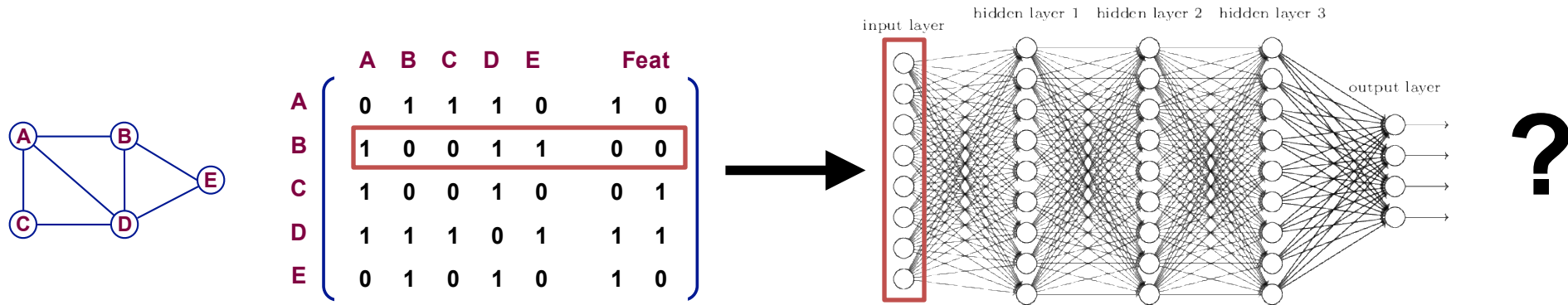
Switching the order of the input leads to different outputs!



Graph Neural Network Overview

Are other neural network architectures, e.g., MLPs, permutation invariant / equivariant?

■ **No.**

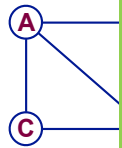


This explains why **the naïve MLP approach fails for graphs!**

Graph Neural Network Overview

Are other
invariant

■ **No.**



Next: Design graph neural networks
that are permutation invariant /
equivariant by passing and
aggregating information from
neighbors!

?

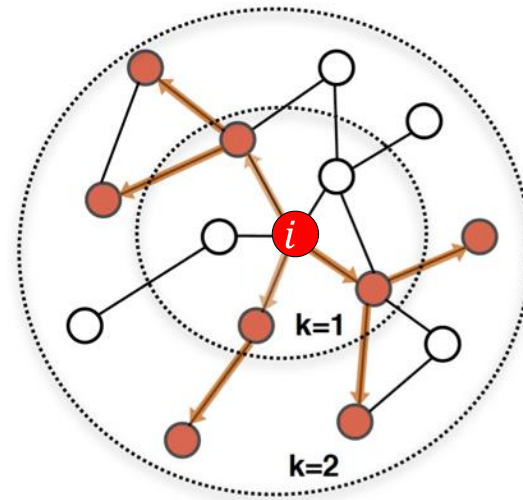
falls for graphs!

Graph Neural Networks: Perspective

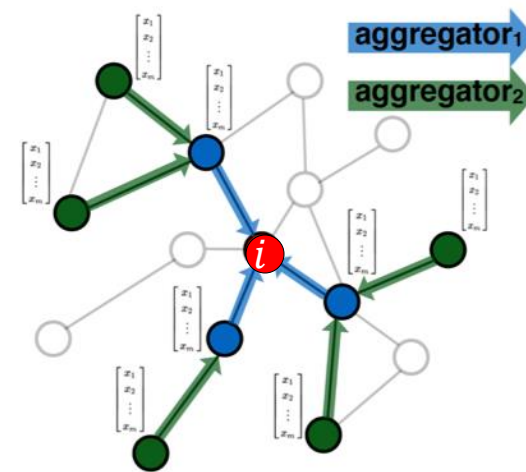
Graph Convolutional Networks

Graph Convolutional Networks

- **Idea:** Node's neighborhood defines a computation graph



Determine node
computation graph

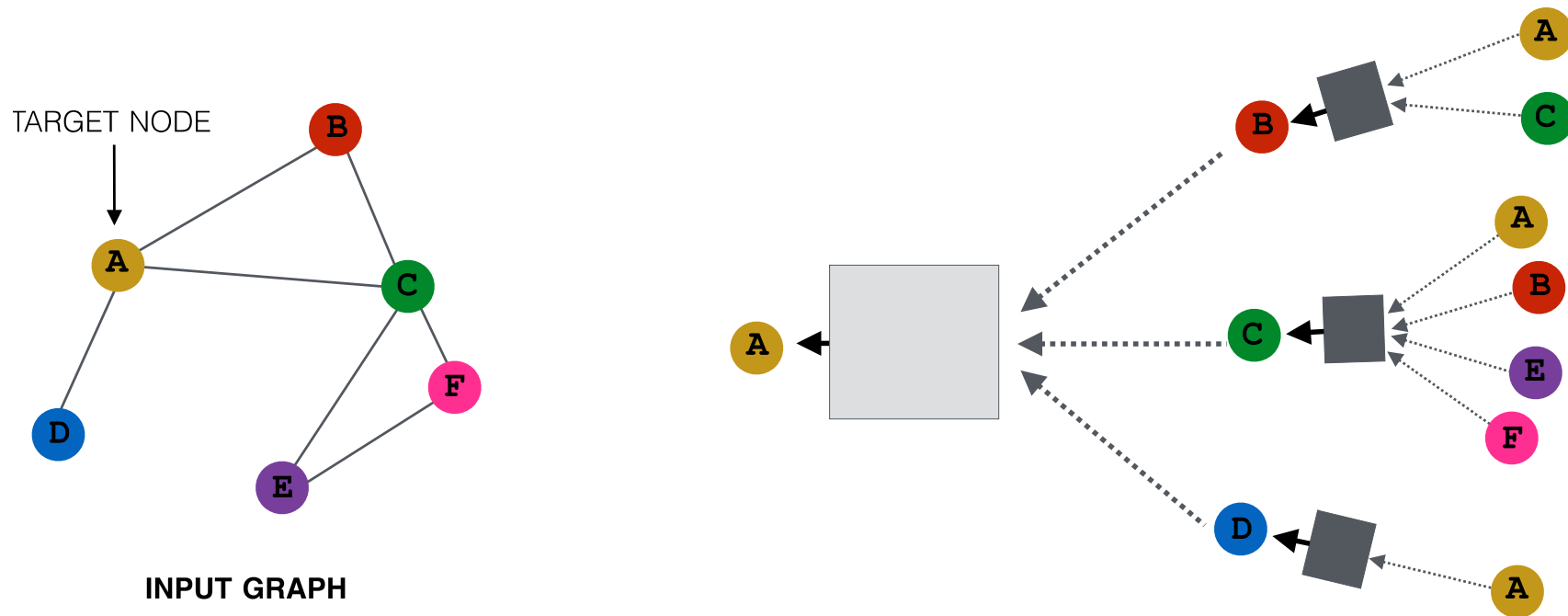


Propagate and
transform information

**Learn how to propagate information across the graph
to compute node features**

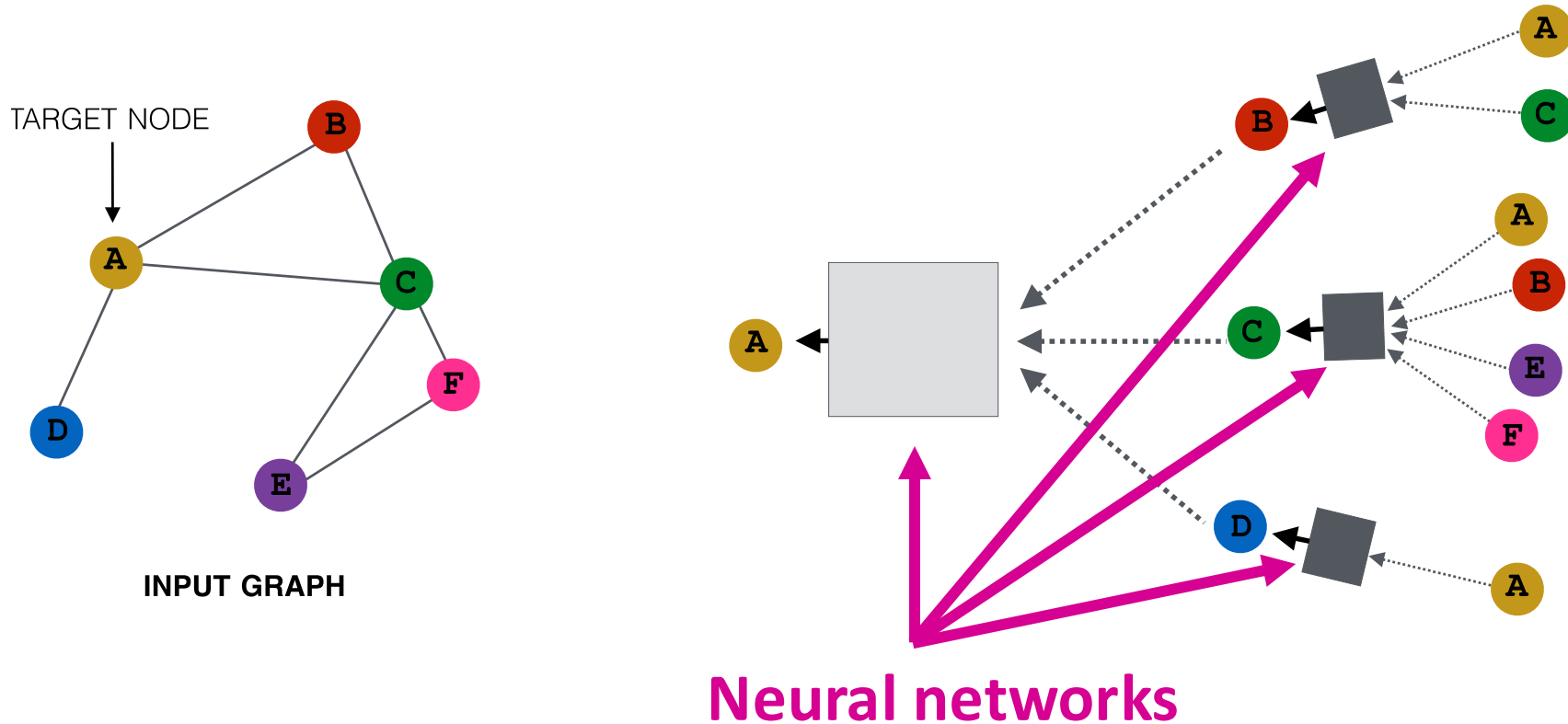
Idea: Aggregate Neighbors

- **Key idea:** Generate node embeddings based on **local network neighborhoods**



Idea: Aggregate Neighbors

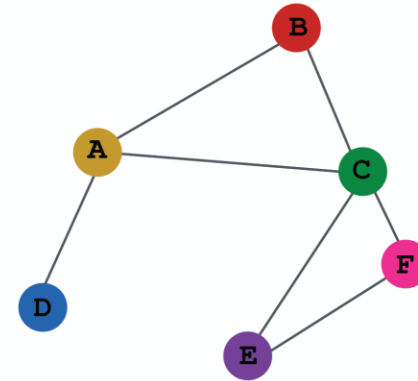
- **Intuition:** Nodes aggregate information from their neighbors using neural networks



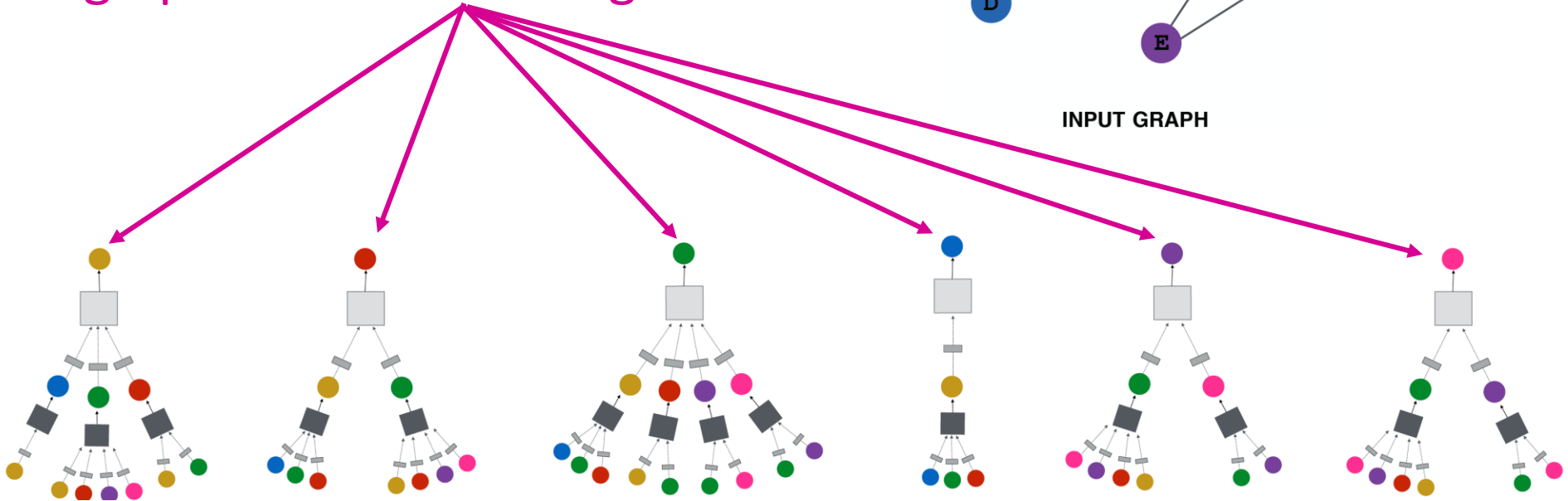
Idea: Aggregate Neighbors

- **Intuition:** Network neighborhood defines a computation graph

Every node defines a computation graph based on its neighborhood!

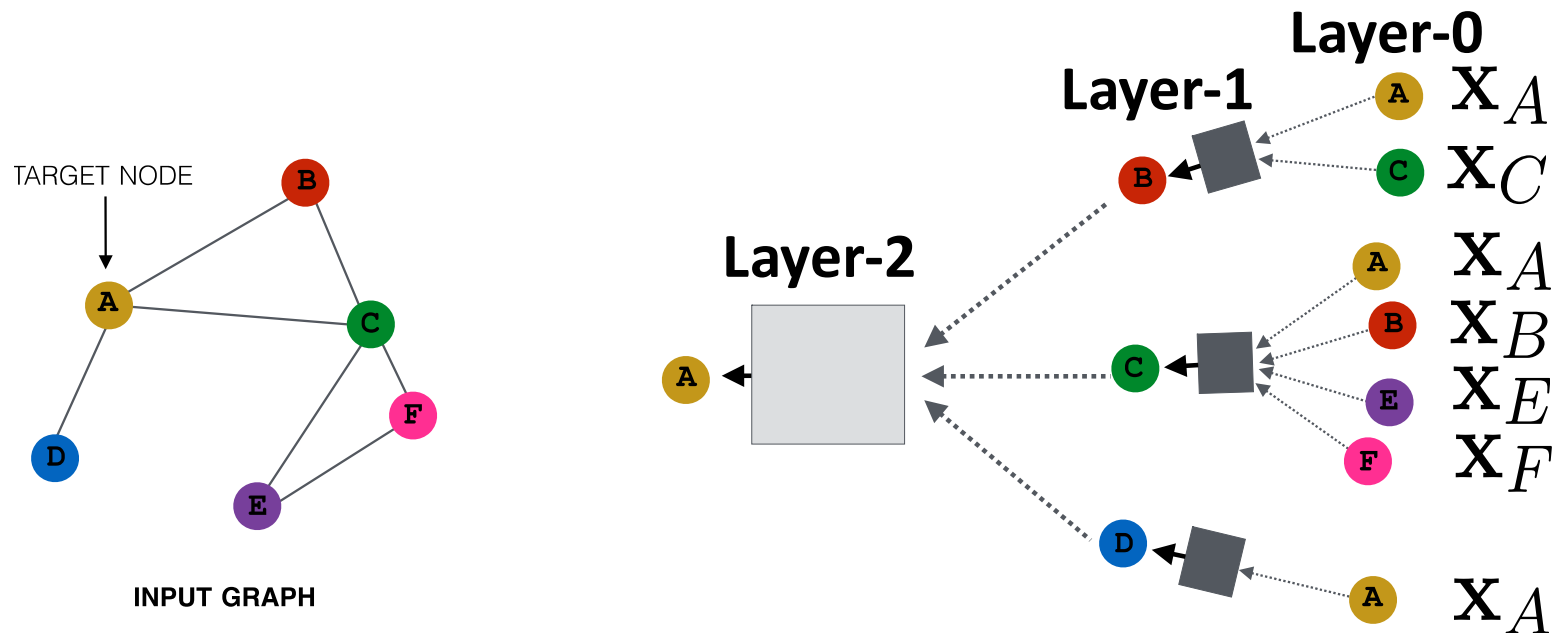


INPUT GRAPH



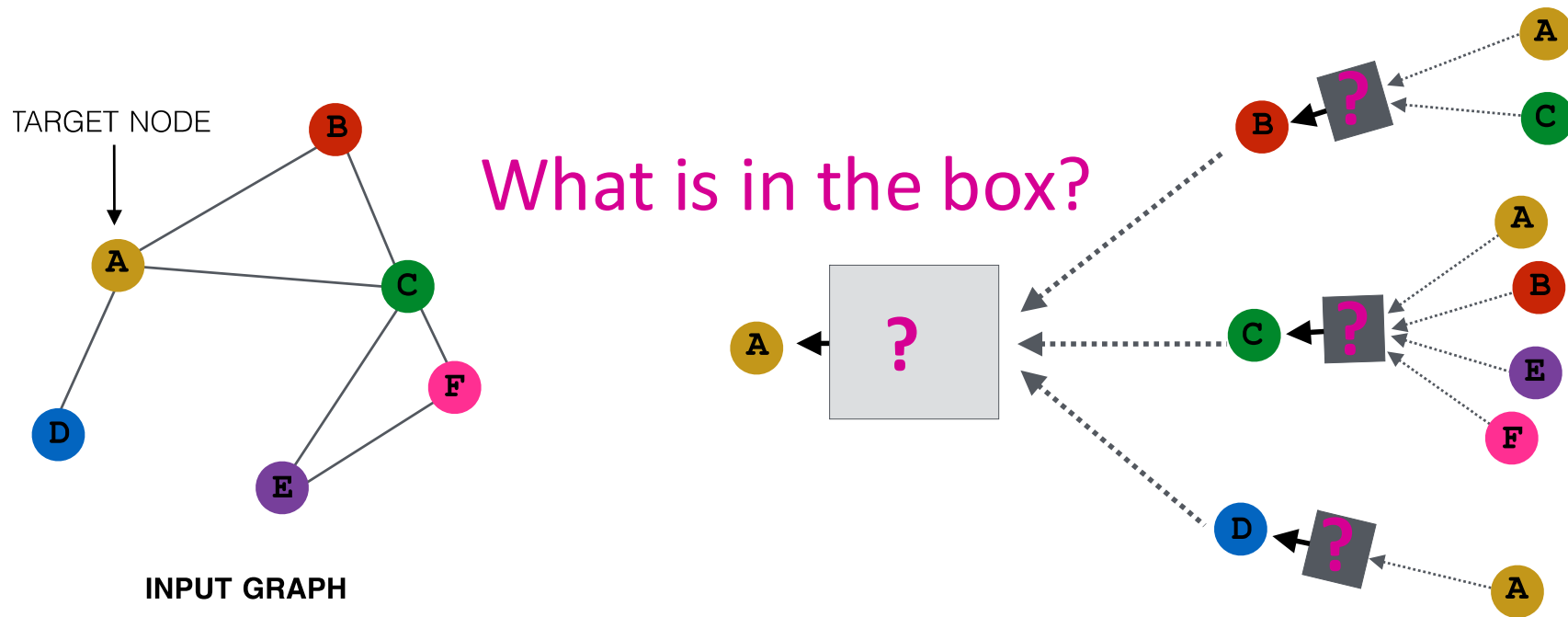
Deep Model: Many Layers

- Model can be of arbitrary depth:
 - Nodes have embeddings at each layer
 - Layer-0 embedding of node v is its input feature, x_v
 - Layer- k embedding gets information from nodes that are k hops away



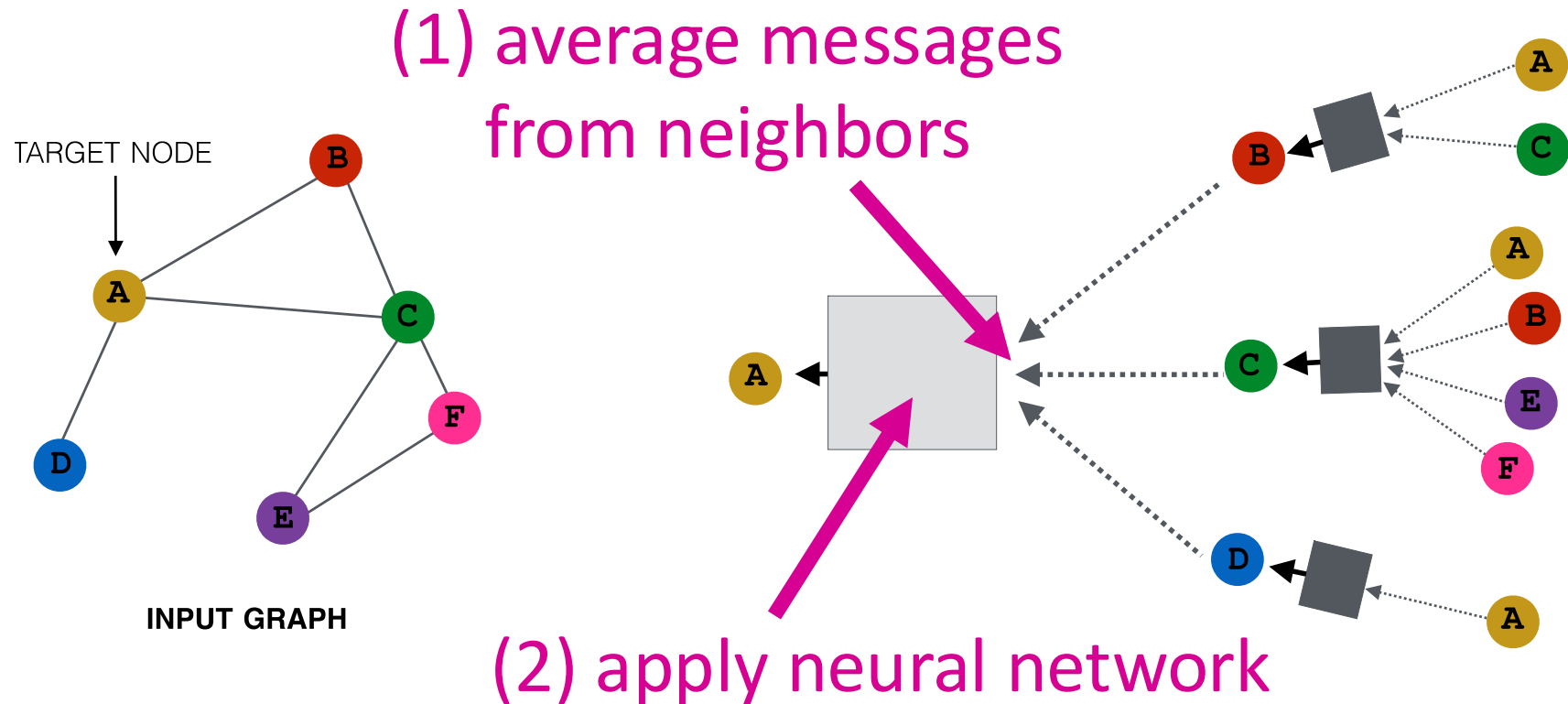
Neighborhood Aggregation

- **Neighborhood aggregation:** Key distinctions are in how different approaches aggregate information across the layers



Neighborhood Aggregation

- **Basic approach:** Average information from neighbors and apply a neural network



The Math: Deep Encoder

- **Basic approach:** Average neighbor messages and apply a neural network

Initial 0-th layer embeddings are equal to node features

$$h_v^0 = x_v$$

embedding of v at layer k

$$h_v^{(k+1)} = \sigma \left(W_k \sum_{u \in N(v)} \frac{h_u^{(k)}}{|N(v)|} + B_k h_v^{(k)} \right), \forall k \in \{0, \dots, K-1\}$$

Average of neighbor's previous layer embeddings

Total number of layers

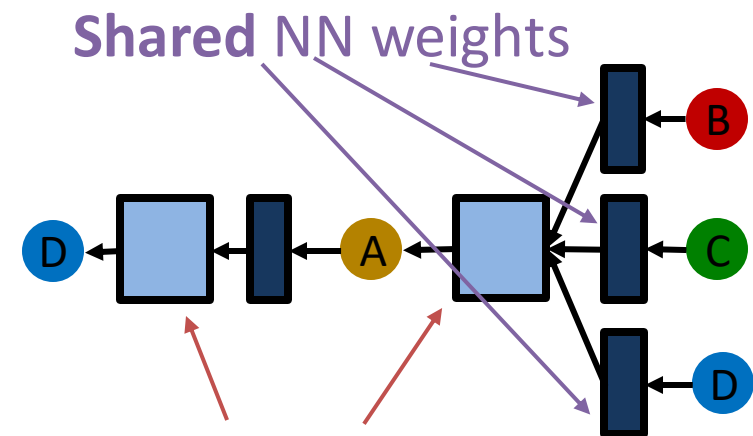
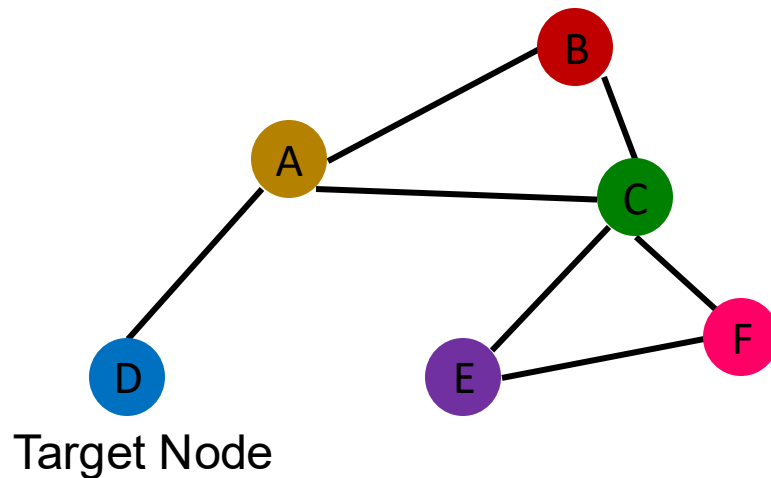
$$z_v = h_v^{(K)}$$

Embedding after K layers of neighborhood aggregation

GCN: Invariance and Equivariance

What are the **invariance** and **equivariance** properties for a GCN?

- **Given a node**, the GCN that computes its embedding is **permutation invariant (output one embedding)**

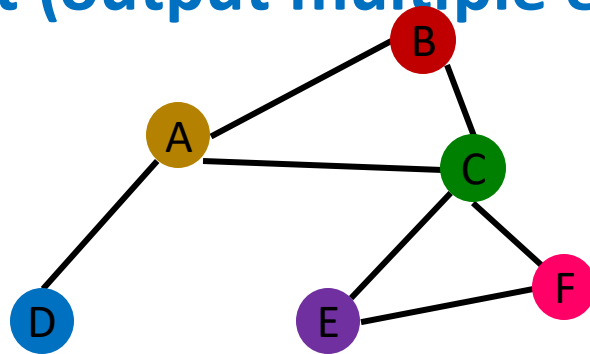


Average of neighbor's previous layer embeddings - Permutation invariant

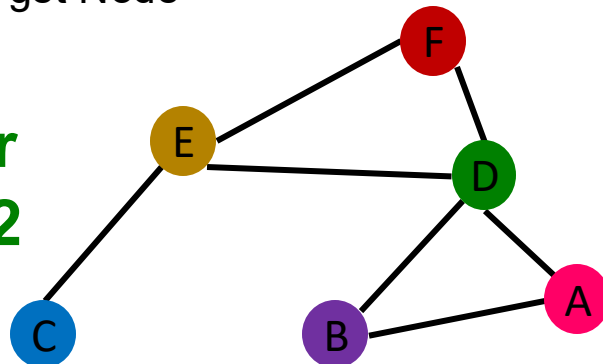
GCN: Invariance and Equivariance

- Considering all nodes in a graph, GCN computation is **permutation equivariant** (output multiple embeddings)

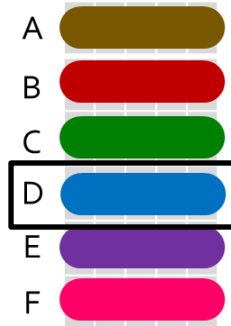
Order
plan 1



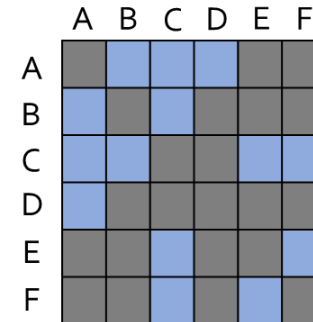
Order
plan 2



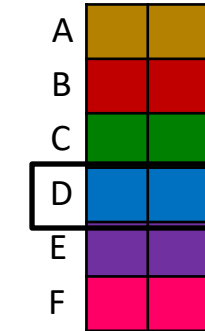
Node feature X_1



Adjacency matrix A_1

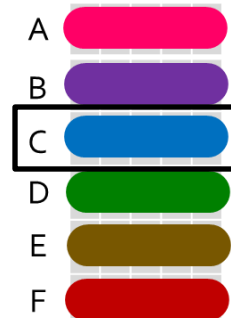


Embeddings H_1

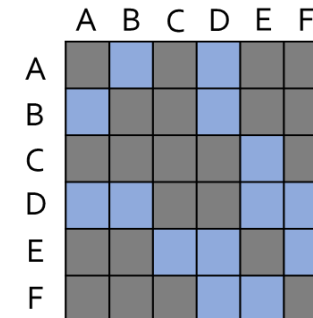


Permute the input, the output also permutes accordingly - permutation equivariant

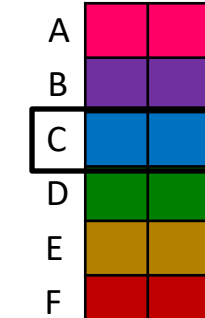
Node feature X_2



Adjacency matrix A_2



Embeddings H_2



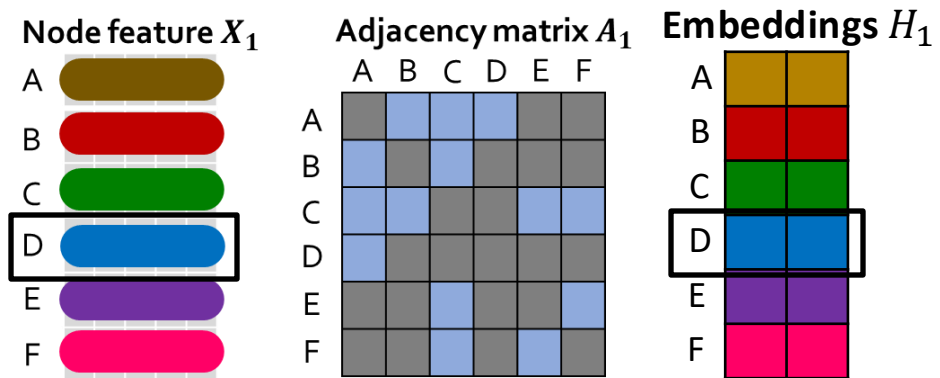
GCN: Invariance and Equivariance

- Considering all nodes in a graph, GCN computation is **permutation equivariant**

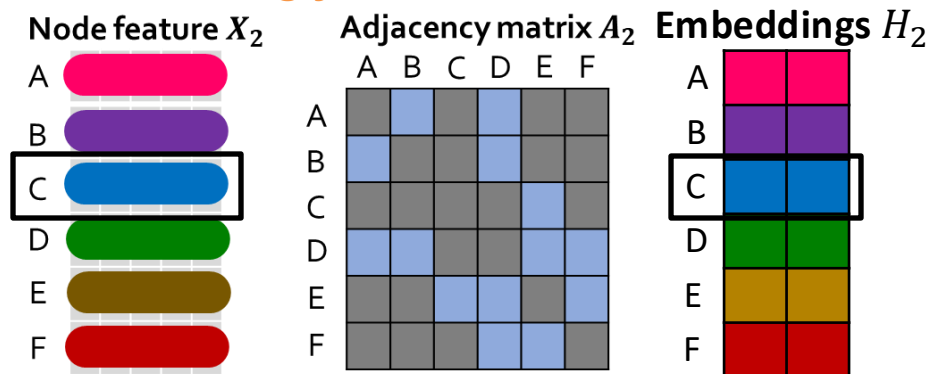
Detailed reasoning:

- The rows of **input node features** and **output embeddings** are **aligned**
- We know computing the embedding of a **given node** with GCN is **invariant**.
- So, after permutation, the **location** of a **given node** in the **input node feature** matrix is changed, and the **the output embedding of a given node stays the same** (the colors of node feature and embedding are **matched**)

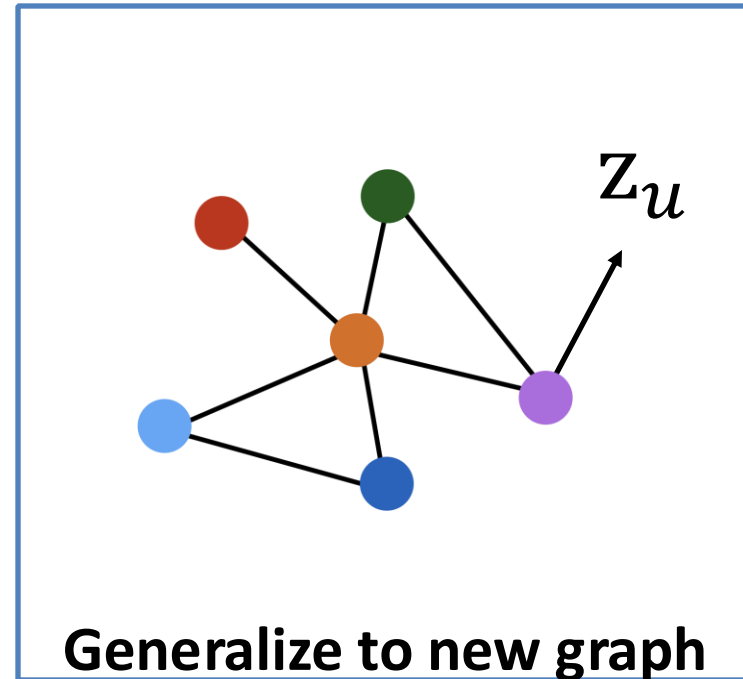
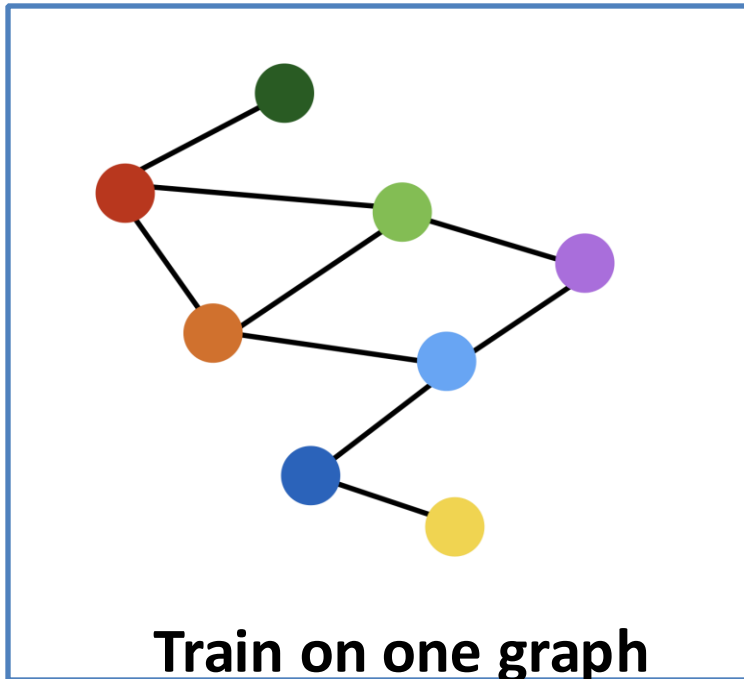
This is permutation equivariant



Permute the input, the output also permutes accordingly - permutation equivariant



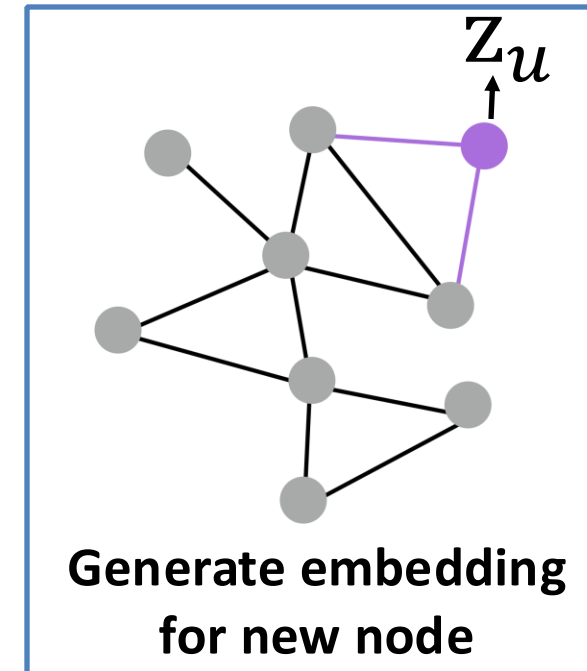
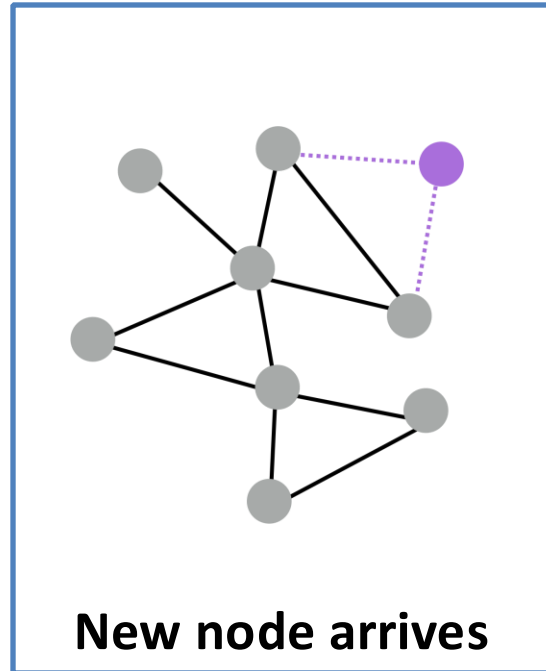
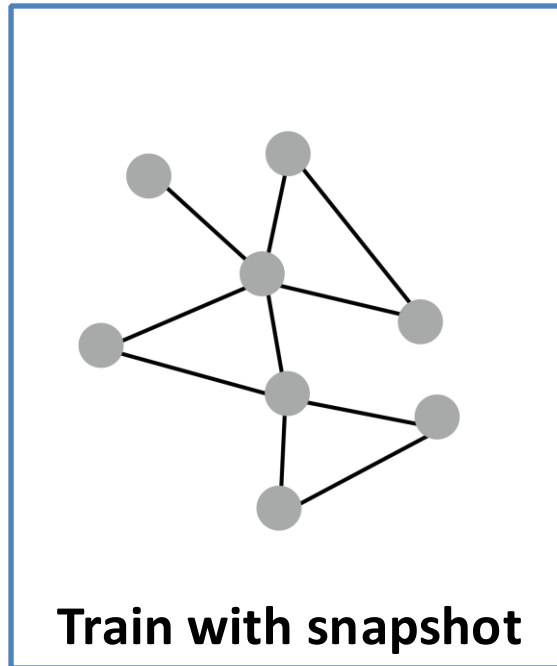
Inductive Capability: New Graphs



Inductive node embedding → Generalize to entirely **unseen graphs**

E.g., train on protein interaction graph from model organism A and generate embeddings on newly collected data about organism B

Inductive Capability: New Nodes



- Many application settings constantly encounter previously **unseen nodes**:
 - E.g., Reddit, YouTube, Google Scholar
- Need to generate new embeddings “on the fly”

Summary

- **In this lecture, we introduced**
 - Idea for Deep Learning for Graphs
 - Multiple layers of embedding transformation
 - At every layer, use the embedding at previous layer as the input
 - Aggregation of neighbors and self-embeddings
 - Graph Convolutional Network
 - Mean aggregation
 - Basics of neural networks - references
 - Loss, Optimization, Gradient, SGD, non-linearity, MLP

Graph Neural Networks: Perspective

Basics of deep learning

Machine Learning as Optimization

- **Supervised learning:** we are given input x , and the goal is to predict label y .
- **Input x can be:**
 - Vectors of real numbers
 - Sequences (natural language)
 - Matrices (images)
 - Graphs (potentially with node and edge features)
- **We formulate the task as an optimization problem.**

Machine Learning as Optimization

- Formulate the task as an optimization problem:

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$$


Objective function

- Θ : a set of **parameters** we optimize
 - Could contain one or more scalars, vectors, matrices ...
 - E.g. $\Theta = \{Z\}$ in the shallow encoder (the embedding lookup)
- $\mathcal{L}$: **loss function**. Example: L2 loss
$$\mathcal{L}(\mathbf{y}, f(\mathbf{x})) = \|\mathbf{y} - f(\mathbf{x})\|_2$$
 - Other common loss functions:
 - L1 loss, huber loss, max margin (hinge loss), cross entropy ...
 - See <https://pytorch.org/docs/stable/nn.html#loss-functions>

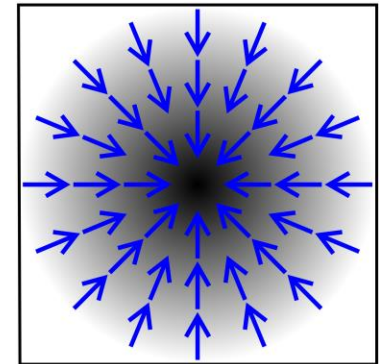
Machine Learning as Optimization

- How to optimize the **objective function**?
- **Gradient vector**: Direction and rate of fastest increase

$$\nabla_{\Theta} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \Theta_1}, \frac{\partial \mathcal{L}}{\partial \Theta_2}, \dots \right)$$

- $\Theta_1, \Theta_2 \dots$: components of Θ
- Recall **directional derivative** of a multi-variable function (e.g. $\mathcal{L}$) along a given vector represents the instantaneous rate of change of the function along the vector.
- **Gradient is the directional derivative in the direction of largest increase.**

Partial derivative



<https://en.wikipedia.org/wiki/Gradient>

Gradient Descent

- **Iterative algorithm:** repeatedly update weights in the (opposite) direction of gradients until convergence

$$\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}$$

- **Training:** Optimize Θ iteratively
 - **Iteration:** 1 step of gradient descent
- **Learning rate (LR) η :**
 - Hyperparameter that controls the size of gradient step
 - Can vary over the course of training (LR scheduling)
- **Ideal termination condition:** gradient = 0
 - In practice, we stop training if it no longer improves performance on **validation set** (part of dataset we hold out from training).

Stochastic Gradient Descent (SGD)

- **Problem with gradient descent:**

- Exact gradient requires computing $\nabla_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$, where $\mathbf{x}$ is the **entire** dataset!
 - This means summing gradient contributions over all the points in the dataset
 - Modern datasets often contain billions of data points
 - Extremely expensive for every gradient descent step

- **Solution: Stochastic gradient descent (SGD)**

- At every step, pick a different **minibatch** $\mathcal{B}$ containing a subset of the dataset, use it as input $\mathbf{x}$

Minibatch SGD

- **Concepts:**
 - **Batch size:** the number of data points in a minibatch
 - E.g. number of nodes for node classification task
 - **Iteration:** 1 step of SGD on a minibatch
 - **Epoch:** one full pass over the dataset (# iterations is equal to ratio of dataset size and batch size)
- **SGD is unbiased estimator of full gradient:**
 - But there is no guarantee on the rate of convergence
 - In practice often requires tuning of learning rate
- Common optimizer that improves over SGD:
 - Adam, Adagrad, Adadelta, RMSprop ...

Neural Network Function

- **Objective:** $\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$
- In deep learning, function f can be very complex
- **Example:**
 - To start simple, consider linear function
$$f(\mathbf{x}) = \mathbf{W} \cdot \mathbf{x}, \quad \Theta = \{\mathbf{W}\}$$
 - Then, if f returns a scalar, then $\mathbf{W}$ is a learnable **vector**
$$\nabla_{\mathbf{W}} f = \left(\frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}, \frac{\partial f}{\partial w_3} \dots \right)$$
 - But, if f returns a vector, then $\mathbf{W}$ is the **weight matrix**

$$\nabla_{\mathbf{W}} f = \begin{bmatrix} \frac{\partial f_1}{\partial w_{11}} & \frac{\partial f_1}{\partial w_{12}} \\ \frac{\partial f_2}{\partial w_{21}} & \frac{\partial f_2}{\partial w_{22}} \end{bmatrix}$$

Jacobian
matrix of f

Intuition: Back Propagation

- **Goal:** $\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$
 - To minimize $\mathcal{L}$, we need to evaluate the gradient: $\nabla_{\mathbf{W}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_1}, \frac{\partial \mathcal{L}}{\partial w_2}, \frac{\partial \mathcal{L}}{\partial w_3} \dots \right)$
which means we need to derive derivative of $\mathcal{L}$.
- **Overview of Back-propagation:**
 - $\mathcal{L}$ is composed from some set of predefined building block functions $g(\cdot)$
 - For each such g we also have its derivative g'
 - Then we can automatically compute $\nabla_{\Theta} \mathcal{L}$ by evaluating appropriate funcs. g' on the minibatch $\mathcal{B}$.

Back-propagation

- How about a more complex function:

$$f(\mathbf{x}) = W_2(W_1\mathbf{x}), \Theta = \{W_1, W_2\}$$

In other words:

$$f(\mathbf{x}) = W_2(W_1\mathbf{x})$$

$$h(x) = W_1\mathbf{x}$$

$$g(z) = W_2z$$

- Recall **chain rule**:

$$\frac{df}{dx} = \frac{dg}{dh} \cdot \frac{dh}{dx} \quad \text{or} \quad f'(x) = g'(h(x))h'(x)$$

- Example: $\nabla_{\mathbf{x}} f = \frac{\partial f}{\partial (W_1\mathbf{x})} \cdot \frac{\partial (W_1\mathbf{x})}{\partial \mathbf{x}}$

- Back-propagation**: Use of **chain rule** to propagate gradients of intermediate steps, and finally obtain gradient of $\mathcal{L}$ w.r.t. Θ .

Back-propagation Example (1)

- **Example:** Simple 2-layer linear network

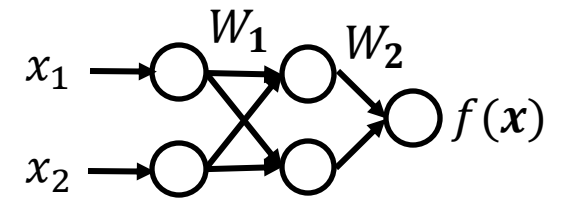
- $f(\mathbf{x}) = g(h(\mathbf{x})) = W_2(W_1\mathbf{x})$

- $\mathcal{L} = \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{B}} \left\| (\mathbf{y}, -f(\mathbf{x})) \right\|_2$

- The loss $\mathcal{L}$ sums the L2 loss in a minibatch $\mathcal{B}$.

- **Hidden layer:**

- Intermediate representation of input $\mathbf{x}$
 - Here we use $h(\mathbf{x}) = W_1\mathbf{x}$ to denote the hidden layer
 - $f(\mathbf{x}) = W_2h(\mathbf{x})$



Back-propagation Example (2)

- Forward propagation:

Compute loss starting from input



- Back-propagation to compute gradient of

$$\Theta = \{W_1, W_2\}$$

Start from loss, compute the gradient

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial \mathcal{L}}{\partial f} \cdot \frac{\partial f}{\partial W_2}$$

Compute backwards

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial \mathcal{L}}{\partial f} \cdot \frac{\partial f}{\partial W_2} \cdot \frac{\partial W_2}{\partial W_1}$$

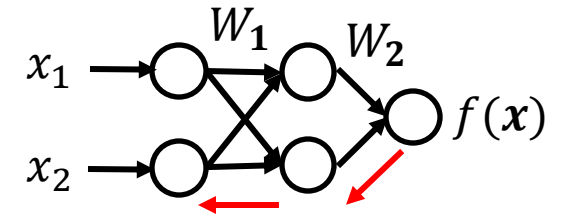
Compute backwards

Remember:

$$f(x) = W_2(W_1 x)$$

$$h(x) = W_1 x$$

$$g(z) = W_2 z$$



Non-linearity

- Note that in $f(\mathbf{x}) = W_2(W_1\mathbf{x})$, W_2W_1 is another matrix (vector, if we do binary classification)
 - Hence $f(\mathbf{x})$ is still linear w.r.t. $\mathbf{x}$ no matter how many weight matrices we compose

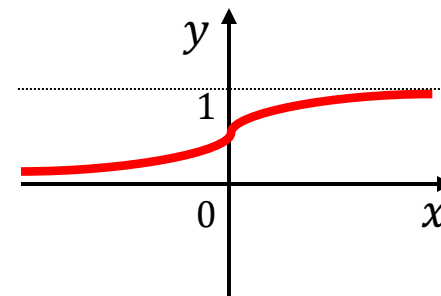
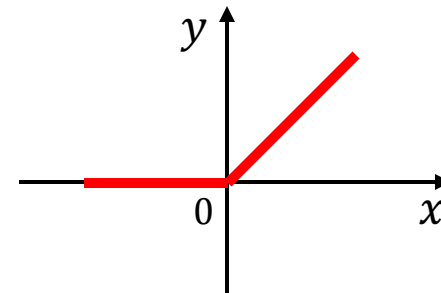
- **We introduce non-linearity:**

- **Rectified linear unit (ReLU)**

$$\text{ReLU}(x) = \max(x, 0)$$

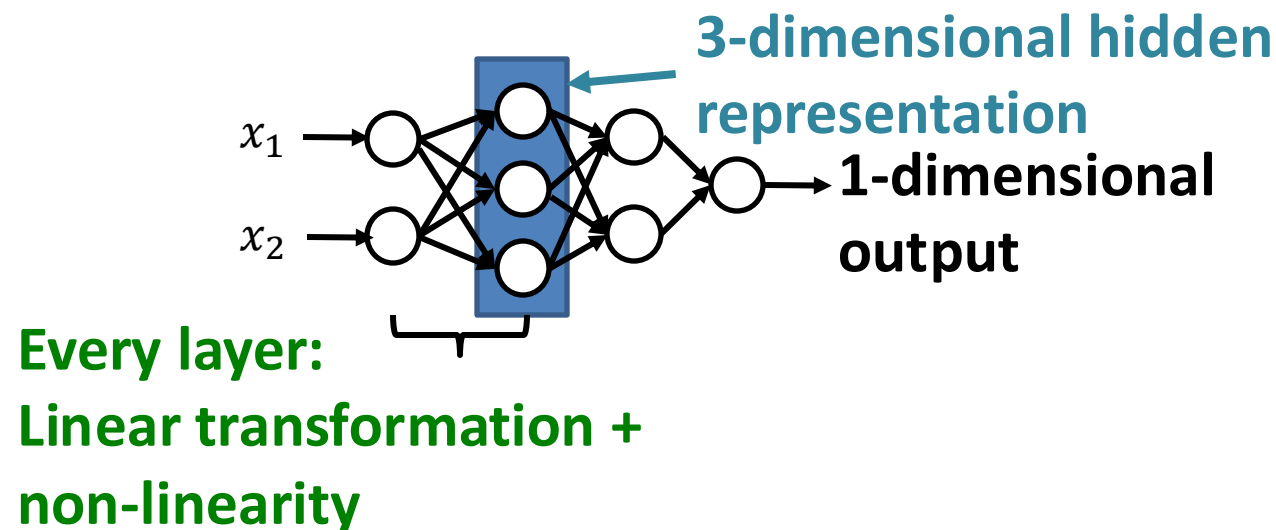
- **Sigmoid**

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Multi-layer Perceptron (MLP)

- Each layer of MLP combines linear transformation and non-linearity:
 - where W_l is weight matrix that transforms hidden representation at layer l to layer $l + 1$
 - b^l is bias at layer l , and is added to the linear transformation of $x^{(l)}$
 - σ is non-linearity function (e.g., sigmoid)
- Suppose x is 2-dimensional, with entries x_1 and x_2



Summary

- **Objective function:**

$$\min_{\Theta} \mathcal{L}(\mathbf{y}, f(\mathbf{x}))$$

- f can be a simple linear layer, an MLP, or other neural networks (e.g., a GNN later)
- Sample a minibatch of input $\mathbf{x}$
- **Forward propagation:** Compute $\mathcal{L}$ given $\mathbf{x}$
- **Back-propagation:** Obtain gradient $\nabla_{\mathbf{w}} \mathcal{L}$ using a chain rule.
- Use **stochastic gradient descent (SGD)** to optimize for Θ over many iterations.